

Reconnecting the phone to App Inventor

If you disconnect the phone and close the Blocks Editor and the App Inventor site, you have to perform quite a few steps when you want to return to working with App Inventor. Here's a quick check-list:

1. Open the App Inventor site, logging in if required.
2. Open the project you were working on, or create a new one. In App Designer, click the 'Open Blocks Editor' button and wait while it downloads and runs the editor.
3. Re-connect the phone via the USB cable (duh!).
4. Check that your USB networking interface (`usb0.0` or, in my case, `eth1`) has an IP address—run `ifconfig` in a terminal; if it shows the interface without an IP address, check your networking settings. If NetworkManager is not set to connect automatically, left-click the notification-area NM icon, and enable the Wired connection for the FreeRunner.
5. Run `adb kill-server` followed by `adb get-state` and you should see something like the following:

```
* daemon not running; starting it now *
* daemon started successfully *
```
6. Now click the Connect to Phone button in the Blocks Editor, and shortly, you should see the preview screen come up on the phone, and the Blocks Editor button caption changing to 'Restart Phone App'.
7. If this previous step fails with an error message from the Blocks Editor, as it did infrequently in my case, I have found that pinging the phone (`ping 192.168.0.202`) and letting around 8 replies come back before hitting Ctrl+C makes the Blocks Editor 'see' the phone. By trial and error, while getting the phone connected the first time, I was again doing data transfer between phone and PC by retrieving several hundred lines of log data, with the `adb shell 'logcat'` command. Thus, it may be some peculiarity with my set-up, phone, or Android on FreeRunner, that requires some data to flow between phone and PC before the Blocks Editor can communicate with the phone.

then re-opening the same project you had opened. This will sometimes cause the Blocks Editor to update and display the blocks for your project.

Returning to Figure 5, on the left you see two tabs (palettes) of blocks: *Built-In* and *My Blocks*. The first of the two contains App Inventor's built-in blocks for use in applications. These include:

- **Definition:** The skeleton blocks of a procedure with a returned result value (commonly called a function); a procedure with no return value; a global variable (at present, local variables are not available, but the Inventor team has accepted a feature request for these, and it should probably soon be available); a *name* for a parameter block to a procedure, which specifies by what name the corresponding parameter's value is available for use inside the procedure; and a *call* (pipe) block, which calls a function and discards the returned value.
- **Text:** Blocks for several common string operations, including slicing/splitting, sub-string searches, concatenation, and equality checks.
- **Lists:** Several common array-handling operations
- **Math:** Blocks for building the Starship Enterprise :-)) (No, just kidding, they're the mathematical operation blocks one would obviously expect from the name.)
- **Logic:** Boolean logic blocks
- **Control:** Flow-control blocks, including *if*, *if-else*, *for each*, *while*, and *close screen* (i.e., close the application,

since each app has only a single screen).

- **Colors:** Pre-defined colour blocks that you can snap to the colour properties of other blocks, which will affect the colour of the component as displayed on the screen.

When you drag a component from the palette to the viewer in the AIA Web application, the Blocks Editor creates corresponding blocks for that component. For example, if you drag a Button component to an otherwise empty viewer in the AIA Web application, in the Blocks Editor you will find newly-added blocks for the button's properties, including *Alignment*, *BackgroundColor*, *Height*, *Width*, *Image* (image to display on the button), *Text* (caption), *TextColor*, and more, as well as event-handler blocks such as *Click()*, *GotFocus()* and *LostFocus()*. These blocks are contained in the *My Blocks* tab of the blocks palette, divided into drawers for each component added in the AIA Web application.

Figure 6 shows the *My Blocks* tab, with the drawer for the button named *ResetButton* opened. The green blocks, '*when ResetButton.Click*' (and *GotFocus* and *LostFocus*) are event-handler blocks. The light-blue blocks are for property gets (read the value of the property) and the darker blue ones with a socket are property set blocks—they assign a new value to the property, based on what you snap into the socket. (Note that the drawer background is actually semi-transparent, and the blocks 'behind' on the canvas show through, giving a cluttered feel; I moved the blocks out of the way in order to give the drawer a clean background in this image capture.)

You can drag, for example, the '*when ResetButton.Click*' block out onto the canvas. To define what happens when you click this button in your app's interface, you would plug into it other blocks representing logical constructs, other components' properties (setters and getters), method invocations, etc. Figure 7 shows the blocks I've filled into this event handler. (Note that there are more blocks than there should be according to the tutorial, because I was playing around to add the extra features suggested at the end of the tutorial.) If the text in the image is visible to you in its printed size, here is what the blocks used in the event handler do:

1. Set the global variable *score* to 0 (represented by a number block with the value 0).
2. Set the global variable *misses* to 0.
3. Reset the timer's interval to the default one second (I had defined behaviour to reduce the interval as the score increased—every 50 points, to play a sound, set the score label's background colour to red, and decrease the timer's interval).
3. Reset the score label's background colour to white, in case the user hits Reset just after a 50-point 'speed-up' score, and the label was red.
4. Call a procedure named *UpdateScore*, whose basic function is to update the text displayed on the labels to the current values of the global variables *score* and *misses* (which we have just set to zero).

Oh, wait—the recycle bin at the bottom right of the Blocks